■ FPGA-Based Wild Animal Detection System

Complete Verilog Architecture Plan

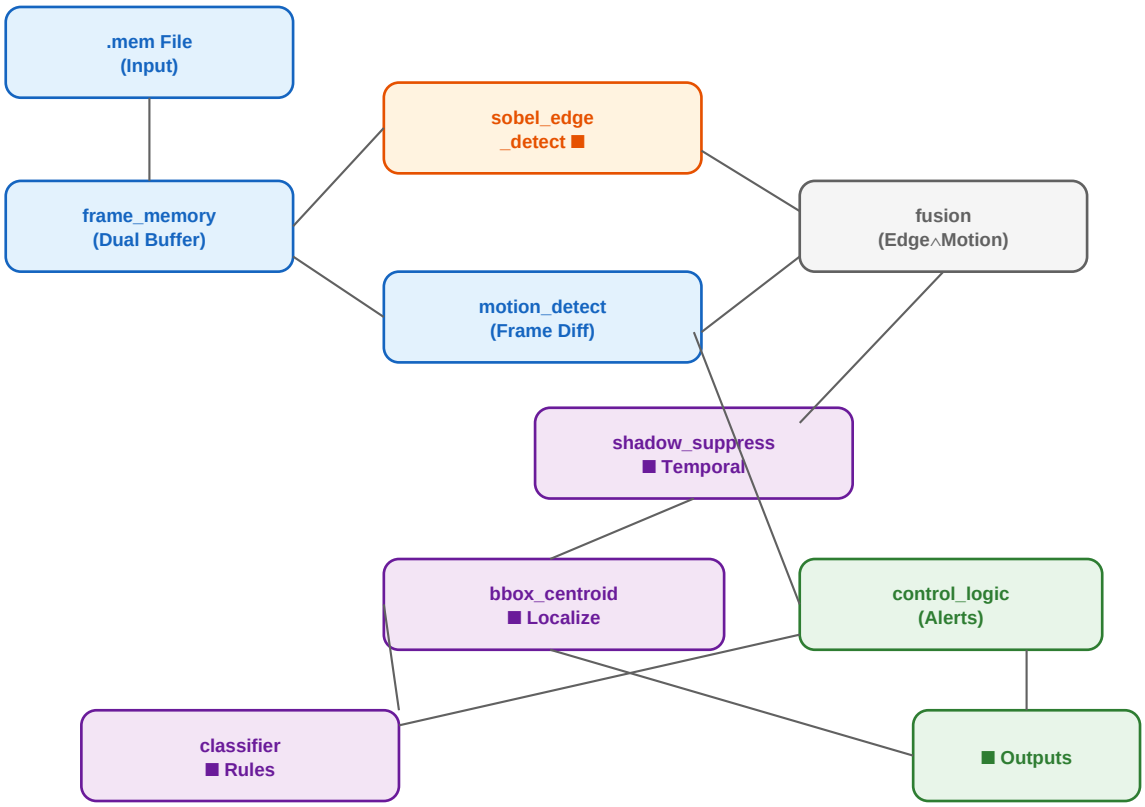
Base Paper: Abhishek & Megalingam, ICCPCT 2025
Target: Xilinx Artix-7 (xc7a35tcpg236-1) · Vivado · 50 MHz
Frame: 128×72 grayscale (9,216 pixels/frame)

This document describes the complete modular Verilog architecture for an FPGA-based wild animal detection system. It reproduces the base paper's pipeline and adds **four novelty improvements**: parallel pipelined Sobel, shadow/false-detection suppression, bounding box + centroid computation, and a lightweight rule-based classifier.

■ ***Novelty 1*** *— Parallel Pipelined Sobel (line-buffer, 1 px/clk)*
■ ***Novelty 2*** *— Shadow / False-Detection Suppression (temporal persistence)*
■ ***Novelty 3*** *— Bounding Box + Centroid Detection (localization)*
■ ***Novelty 4*** *— Lightweight Animal-vs-Non-Animal Classifier (rule-based)*

1. System Architecture — Full Pipeline

The system processes 128×72 grayscale video frames loaded from .mem files. Pixels are streamed through the pipeline at one pixel per clock cycle after initial fill. The diagram below shows the complete data flow:



Pipeline flow: *Input Frame → frame_memory → [parallel Sobel ■ + motion_detect] → fusion → shadow_suppress ■ → bbox_centroid ■ → classifier ■ → control_logic → Alerts*

2. Module Breakdown

2.1 frame_memory — Dual-Frame Buffer

Stores the current frame and retains the previous frame for motion comparison. Uses two Block RAM arrays of 9,216 × 8-bit each. On loading a new frame, the current frame is copied to the previous buffer, then overwritten with incoming data. The previous frame is zeroed on reset (first frame has no reference).

Port	Dir	Width	Description
clk	in	1	System clock
rst	in	1	Synchronous reset
load_en	in	1	Pulse high to load a new frame
pixel_in	in	8	Incoming pixel data
pixel_addr	in	14	Write address (0..9215)
frame_ready	out	1	High when full frame loaded
curr_pixel	out	8	Read: current frame pixel
prev_pixel	out	8	Read: previous frame pixel
rd_addr	in	14	Read address for both ports

2.2 sobel_edge_detect — Parallel Sobel ■ Novelty #1

Computes Sobel gradient magnitude with **parallel Gx/Gy computation** and a **pipelined 3×3 line-buffer** architecture. Two 128-pixel line buffers form a sliding 3×3 window. Gx and Gy kernels are applied simultaneously in pipeline stage 1. Stage 2 computes |Gx| + |Gy| (Manhattan approximation). Stage 3 applies the threshold. Achieves 1 pixel/clock throughput after 2-line + 3-stage pipeline fill.

> ***Key novelty vs. base paper:*** *The base paper computes Sobel on full frames stored in memory. Our version streams pixels with only 2 lines of buffer, enabling much higher throughput.*

Port	Dir	Width	Description
clk	in	1	System clock
rst	in	1	Synchronous reset
pixel_in	in	8	Streaming pixel input
pixel_valid	in	1	Input valid strobe
edge_threshold	in	8	Gradient magnitude threshold
edge_pixel	out	1	1 = edge, 0 = not edge
edge_valid	out	1	Output valid (pipeline latency)
edge_magnitude	out	16	Raw |Gx|+|Gy| for fusion weighting

```
Sobel Kernels (computed in parallel):
Gx = [-1 0 +1]    Gy = [-1 -2 -1]
     [-2 0 +2]          [ 0 0 0]
     [-1 0 +1]          [+1 +2 +1]
```

2.3 motion_detect — Frame Differencing

Generates a binary motion map by computing |curr_pixel - prev_pixel| for each pixel. If the absolute difference exceeds the configurable motion threshold, the pixel is marked as 'motion'. An accumulator counts total motion pixels per frame.

Port	Dir	Width	Description
curr_pixel	in	8	Current frame pixel
prev_pixel	in	8	Previous frame pixel
pixel_valid	in	1	Valid strobe
motion_threshold	in	8	Min intensity difference
motion_pixel	out	1	1 = motion, 0 = static
motion_valid	out	1	Output valid
motion_count	out	14	Running count of motion pixels

2.4 fusion — Edge + Motion Merge

Combines the Sobel edge map and the motion map to produce a fused foreground map. Default: AND fusion (pixel must be both an edge AND moving). Timing alignment is handled by inserting delay registers on the motion path to match Sobel's ~3-cycle pipeline latency.

Port	Dir	Width	Description
edge_pixel	in	1	From Sobel module
motion_pixel	in	1	From motion detect
edge_valid	in	1	Edge data valid
motion_valid	in	1	Motion data valid
fused_pixel	out	1	Fused foreground pixel
fused_valid	out	1	Output valid

2.5 shadow_suppress — Temporal Persistence ■ Novelty #2

Suppresses shadows, flicker, and transient false detections by maintaining a per-pixel persistence counter. Each pixel has a 3-bit counter stored in BRAM. When a pixel is detected, its counter increments (saturating at max). When not detected, the counter resets to 0. Only pixels whose counter reaches the configured threshold (default: 3 frames) are accepted.

> ***Directly addresses Testcase 3*** *from the base paper where shadows near a vehicle caused false positives. With persistence threshold = 3, a shadow lasting only 1–2 frames is filtered.*

Port	Dir	Width	Description
fused_pixel	in	1	Input from fusion
pixel_addr	in	14	Pixel address in frame
frame_done	in	1	Pulse at end of each frame
persist_thresh	in	3	Consecutive frames required (2–7)
stable_pixel	out	1	1 = stable (real) detection

2.6 bbox_centroid — Bounding Box + Centroid ■ Novelty #3

Computes the bounding box (Xmin, Xmax, Ymin, Ymax) and centroid (Cx, Cy) of the detected foreground region. During the frame scan, for each stable pixel: min/max registers track bounding box extremes while accumulators sum X, Y coordinates and count. On frame_done, centroid = (sum_x/count, sum_y/count) via a sequential divider.

Port	Dir	Width	Description
stable_pixel	in	1	From shadow suppression
pixel_x	in	7	Column (0..127)
pixel_y	in	7	Row (0..71)
pixel_valid	in	1	Valid strobe
frame_done	in	1	End-of-frame pulse
x_min, x_max	out	7	Bounding box horizontal
y_min, y_max	out	7	Bounding box vertical
centroid_x	out	7	Centroid X coordinate
centroid_y	out	7	Centroid Y coordinate
pixel_count	out	14	Total foreground pixels
bbox_valid	out	1	Results valid after frame_done

2.7 classifier — Lightweight Animal Classifier ■ Novelty #4

Rule-based classification using bounding box geometry and motion features. No CNN or ML model required — purely combinational comparators and multipliers.

Classification Rules (all must pass):

- **Rule 1:** min_width ≤ width ≤ max_width
- **Rule 2:** min_height ≤ height ≤ max_height
- **Rule 3:** pixel_count ≥ min_area
- **Rule 4:** aspect_ratio = (width×16)/height ≤ max_aspect_ratio (fixed-point)
- **Rule 5:** fill_ratio: pixel_count × 4 ≥ width × height (≥25% fill, avoids division)

Port	Dir	Width	Description
bbox_valid	in	1	Trigger from bbox module
width / height	in	7	Bounding box dimensions
pixel_count	in	14	Foreground area
motion_count	in	14	Total motion pixels
min/max_width	in	7	Configurable bounds
min/max_height	in	7	Configurable bounds
min_area	in	14	Minimum foreground pixels
max_aspect_ratio	in	8	Max W/H × 16 (fixed-point)
is_animal	out	1	Classification result
class_valid	out	1	Output valid

2.8 control_logic — Alert Generation

Combines motion detection and classification results to generate final alert signals. motion_alert goes high when any motion pixels exist. animal_alert goes high when the classifier outputs is_animal=1 AND motion is detected. Both are registered outputs updated once per frame.

Port	Dir	Width	Description
motion_detected	in	1	Any motion in frame
is_animal	in	1	Classifier output
class_valid	in	1	Classifier done
motion_alert	out	1	Motion alert output
animal_alert	out	1	Animal detection alert

2.9 top — Top-Level Integration

Wires all 8 sub-modules together. Contains a frame-scanner FSM that generates sequential pixel addresses and (x, y) coordinates, manages frame loading, pipeline synchronization (delay registers aligning Sobel and motion outputs), and frame_done signaling.

FSM States: IDLE → LOAD_FRAME → SCAN → WAIT_RESULTS → OUTPUT → (loop)

3. File Organization

```
src/
■■■ rtl/
■ ■■■ top.v                # Top-level integration + FSM
■ ■■■ frame_memory.v     # Dual-frame buffer (BRAM)
■ ■■■ sobel_edge_detect.v # Pipelined Sobel ■
■ ■■■ line_buffer.v      # 128-pixel shift register
■ ■■■ motion_detect.v    # Frame differencing
■ ■■■ fusion.v           # Edge ∧ motion merge
■ ■■■ shadow_suppress.v  # Temporal persistence ■
■ ■■■ bbox_centroid.v    # Bounding box + centroid ■
■ ■■■ classifier.v       # Rule-based classifier ■
■ ■■■ control_logic.v    # Alert generation
■ ■■■ divider.v          # Sequential divider
■■■ tb/
■ ■■■ top_tb.v           # System testbench
■ ■■■ sobel_tb.v         # Sobel unit test
■ ■■■ motion_tb.v        # Motion detect test
■■■ constrs/
■ ■■■ constraints.xdc
■■■ mem/
    ■■■ video_1/           # Test .mem files
```

4. Phased Build Order

Phase	Modules	Test
Phase 1	frame_memory + motion_detect + control_logic + basic top	Verify motion_alert toggles on frames with/without movement
Phase 2	sobel_edge_detect + line_buffer + fusion	Verify edge map on known test patterns (checkerboard)
Phase 3	shadow_suppress	Feed 5+ identical frames → counter reaches threshold

Phase	Modules	Test
Phase 4	bbox_centroid + divider	Feed known foreground → verify coordinates
Phase 5	classifier	Feed various bbox sizes → verify animal/non-animal
Phase 6	Full integration + .mem testbench	Run full video sequences, compare alerts

5. Key Design Parameters

Parameter	Value	Rationale
Frame width	128 px	Matches base paper
Frame height	72 px	Matches base paper
Pixel depth	8 bits	Grayscale
Clock	50 MHz	Base paper specification
Motion threshold	30 (configurable)	~12% of 255; rejects noise
Edge threshold	50 (configurable)	Sobel gradient cutoff
Persistence threshold	3 frames	Suppress 1–2 frame shadows
Min animal size	10×10 px	Smallest expected blob
Max animal size	100×60 px	Largest (nearly full frame)

6. Base Paper vs. Our Novelty — Separation

Component	Base Paper	Our Novelty
Frame memory (dual buffer)	■	
Motion detection (frame diff)	■	
Sobel edge detection (basic)	■	
Control logic / alerts	■	
Parallel pipelined Sobel (line buf)		■
Shadow / false-detection suppression		■
Bounding box + centroid		■
Lightweight rule-based classifier		■
Fusion module (edge + motion)		■

Total modules: *9 Verilog modules + 1 divider helper + 1 Python preprocessing script.*
Build follows strict incremental testing — each module verified in isolation before integration.